

GPT-Reviewed VLA

Keep the Policy or Correct Its End-Effector Action

Research Proposal III · Zimo Wen

September 2026

Abstract

A vision–language–action policy can execute familiar manipulation efficiently, while a general-purpose GPT model can inspect task context and identify errors that the policy may miss. This proposal studies a deployed hybrid agent that retains both components. The VLA proposes an action chunk, and GPT chooses between two execution branches: keep the original action, or construct a bounded end-effector (EEF) correction through Code as Policy and audit it before execution. The same GPT model can perform selection, correction, and review in separate calls. The central question is whether selective correction improves task success without damaging actions the VLA would already execute correctly. We also propose a benchmark with natural rollouts and private, paired checkpoint continuations to separate branch selection, correction quality, unnecessary intervention, and inference cost. GPT remains active during evaluation and deployment; removing it is not the objective.

1. Research Question and the Intended Paper

Deployment mechanism. A VLA proposes behavior from an instruction and current observations. GPT reviews this proposal and the visible scene. If it is appropriate, execution retains the original action. If a local geometric change is needed, GPT writes an EEF correction, reviews that correction, and submits the approved result. After execution, new observations re-enter the loop. This is a persistent review-and-correction system, rather than a training-only teacher whose role disappears after fine-tuning.

Why two branches matter. A reviewer that always intervenes can damage an already capable VLA. A reviewer that always accepts preserves the VLA but cannot rescue mistakes. For example, the proposed grasp may already align with a cup, or it may need a small lateral or orientation adjustment. The research problem is to identify the cases where preserving the proposal is preferable and to generate a useful correction when it is not. Decision quality and correction quality must both be measured.

EEF correction, not policy-weight modification. Here EEF denotes end-effector pose or motion commands. GPT changes a bounded part of the current action trajectory, using explicit position/orientation offsets, local waypoints, or a short feedback program. The main study freezes VLA weights and retains existing low-level controllers. GPT does not introduce a learned force controller or update the VLA’s parameters at each review. The effect on the executed policy comes from selecting and modifying actions at runtime.

Central hypothesis. A persistent GPT reviewer can improve a frozen VLA’s closed-loop success by applying a small number of useful EEF corrections, while retaining most actions that are already effective. The paper should show a favorable success–intervention–latency tradeoff compared with always keeping, always correcting, and matched heuristic gates. It should also identify conditions in which review or self-audit is unreliable.

Relationship to the other proposals and prior work. The first paper studies which control quantities the agent should output; the second studies when changed observations can affect execution. This paper studies **which VLA proposals to trust and how to correct the selected ones**. Code as Policies provides executable programs, while VLA policies provide nominal actions. DAgger-style aggregation is a possible later extension, but is not the main deployment mechanism or evaluation target. A broader literature comparison is needed before asserting a first-of-its-kind contribution.

2. Method: Binary Review with an Audited EEF Correction

Visible input. Let $A_t = \pi_\theta(o_t, g, h_t)$ be the VLA’s proposed short action sequence. GPT receives the instruction, allowed images and proprioception, bounded visible history, and an inspectable representation of A_t . Where a VLA outputs joint commands, a declared adapter may express their intended EEF motion using robot kinematics. It must not inspect hidden object poses or simulator contact truth. The candidate remains associated with its observation timestamp, action version, and coordinate convention.

Binary execution rule. The reviewer produces $b_t \in \{\text{KEEP}, \text{CORRECT}\}$ and a concise, observable justification. For an approved correction program p_t :

$$A_t^{\text{exec}} = \begin{cases} A_t, & b_t = \text{KEEP}, \\ C(A_t, p_t, o_t), & b_t = \text{CORRECT}. \end{cases}$$

C applies an explicitly bounded EEF edit and translates it to the existing action interface. KEEP means the unchanged valid proposal, not a newly generated replacement trajectory. CORRECT edits the declared segment and records what changed. These remain the two task-action branches; waiting for a fresh observation or reporting an invalid proposal is execution management, not an undisclosed third controller.

Correction and audit. In the CORRECT branch, GPT creates a candidate containing a frame, translation/rotation or waypoints, maximum duration, and observable completion conditions. The same GPT snapshot reviews the candidate against the original proposal and scene in a separate call. A deterministic interface check independently validates types, finite values, action bounds, and supported calls. Allow at most a preregistered number of revisions. Rejected or expired corrections are not silently executed, and invalid proposals count against completion and cost. GPT approval is not a simulator success verdict.

3. Three Technical Approaches to the Correction Branch

Approach A: geometric EEF edits. GPT predicts a small translation and rotation offset for a selected VLA segment, with explicit tool- or world-frame semantics. Apply the edit to a trajectory reference and preserve the rest of the valid proposal. This is the simplest interface for testing whether the reviewer can identify misalignment and useful correction direction. Compare with matched fixed or perception-derived offsets; correct units and orientation composition are essential.

Approach B: bounded Code-as-Policy repair. GPT produces a short local program using declared EEF primitives, for example re-observe, move to a corrected approach pose, and resume the skill. The program may use local feedback but has a fixed duration and revision budget. It cannot replace the task with an unrestricted new plan. Retaining the VLA proposal, affected segment, and handoff rule makes the correction inspectable and separates local repair from full GPT takeover.

Approach C: prediction-assisted review. Present GPT with a standardized kinematic preview or observation-conditioned prediction of the original and corrected short motions, including uncertainty. GPT selects or revises the correction using only deployable evidence. Compare with the same reviewer without previews, matching call budgets. A learned preview can be wrong; future simulator states are never shown to the reviewer. This approach tests whether action consequences improve binary judgment beyond inspecting the current image and action numbers.

4. Pipeline, State Validity, and Runtime Boundaries

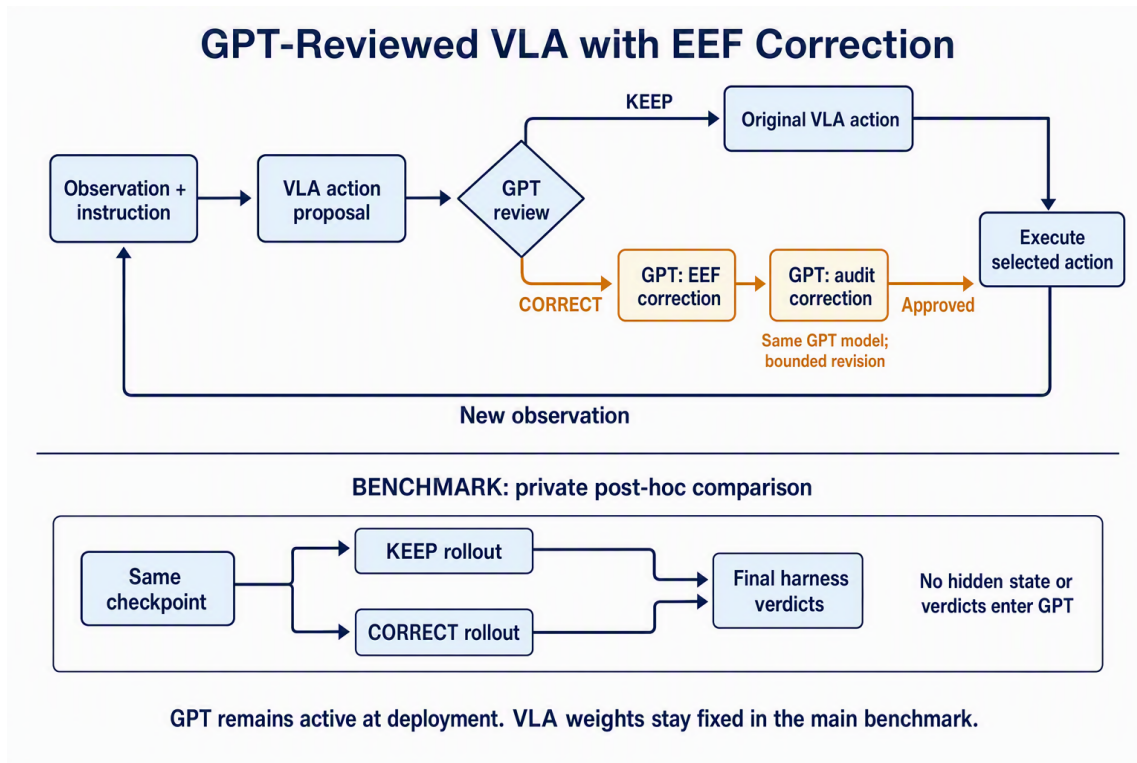


Figure 1: GPT remains in the deployment loop. Review selects KEEP or CORRECT; GPT-generated EEF corrections are audited before execution. The separate benchmark panel compares outcomes privately and never returns hidden state or branch verdicts to GPT. GPT-image-2 conceptual diagram, not a measured result.

One GPT model, several roles. Selection, correction generation, and audit can use the same fixed GPT model and version. They are distinct calls with bounded inputs, not independent sources of evidence. Correlated mistakes and confirmation bias are therefore explicit risks. Compare a single-pass choice with a second-pass audit, while holding total token or call budgets fixed. A model name, including a chosen GPT release, is a configuration to record rather than a guarantee of review accuracy.

Preventing stale edits. Before applying an EEF correction, check the observation and proposal versions. After any intervention, obtain a fresh observation and re-query the VLA; replaying an old, geometrically invalid suffix is prohibited. During review, use a declared hold or validated executable prefix. Environmental evolution and review latency are included in real-time evaluation. A paused-simulation diagnostic must be labeled separately and cannot establish deployment responsiveness.

Existing implementation entry points. Robot-RSI exposes relative EEF motion, pose observation, and servo-based correction paths. These support a concrete adapter but do not establish that arbitrary VLA chunks can already be edited correctly. Validate action scaling, frames, quaternion conventions, gripper semantics, cancellation, and action ownership. Hidden target positions, simulator predicates, and evaluator-owned checkpoints remain outside GPT and generated-code access.

5. Benchmark Design: Selection, Correction, and Full Episodes

Proposed benchmark. We propose *VLA Review Benchmark* as a working title. It evaluates the deployed VLA+GPT system, with frozen VLA and reviewer configurations. The two tracks below distinguish local decision quality from complete robot behavior. The benchmark is initially built on LIBERO short tasks; its checkpoint and disturbance extensions are reported separately from standard task-level scores.

Track A: paired checkpoint continuations. Retain checkpoints from natural VLA rollouts using an evaluator-only mechanism. At a checkpoint, present the same permitted observation and original VLA proposal to each reviewer. Execute the KEEP continuation and the submitted CORRECT continuation in separate simulator branches with matched initial state and exogenous randomness, using the same frozen VLA for subsequent continuation. The agent cannot inspect checkpoint internals or other branch outcomes. Endpoint labels come from final harness verdicts, not a GPT judgment that the EEf pose looks better.

Let $Y_K, Y_C \in \{0, 1\}$ be final success for the unchanged proposal and the submitted correction. The four cases $(0, 1)$, $(1, 0)$, $(1, 1)$, and $(0, 0)$ respectively indicate a beneficial correction, a harmful correction, equal successful outcomes, and unresolved failure. These labels describe the evaluated candidate and continuation, not a universal oracle declaring that a state is correctable by any possible method. For stochastic environments, estimate outcome probabilities over matched continuation seeds rather than relying on one binary result.

Lock the review decision before evaluating either branch. For KEEP decisions, obtain a counterfactual correction in a separate diagnostic call without allowing the decision to change. This avoids measuring correction quality only on states the reviewer selected. Its extra generation and branching costs belong to benchmark construction, not deployed inference cost. Track A isolates checkpoint decisions; Track B includes actual review delay and environmental evolution.

Track B: closed-loop episodes. Deploy the complete reviewer with its real decisions, repeated observations, correction audits, and latency. This tests distribution shift caused by its own interventions. A good checkpoint score does not ensure a good multi-step hybrid policy. Include unperturbed tasks where the VLA already works, local geometric errors that EEf edits may repair, and cases outside the allowed correction envelope. The last group prevents the benchmark from rewarding plausible but ineffective edits; failure remains a legitimate outcome.

Proposed panel and splits. An initial diagnostic test panel contains 24 held-out task-condition pairs with 10 retained checkpoints each: 240 checkpoints. Add a closed-loop panel of the same 24 pairs with five independent initial seeds: 120 episodes per method. Development and validation use different task families and objects. These counts are starting designs, adjusted using development variance; near-duplicate task recipes stay within one split. Release a naturally sampled panel and a separately labeled, stratified stress panel rather than silently balancing outcomes to favor correction.

Configuration and fairness. Freeze model snapshots, prompts, correction bounds, review frequency, revision limits, preview methods, and budgets before testing. Record model-specific sampled checkpoints; do not assume one VLA’s failure states represent another VLA. Compare additional VLA backbones and reviewer models on declared tracks. All algorithms receive the same allowed sensing, action access, and wall-time accounting. The standard 120-task LIBERO panel additionally reports pass@10 by spatial/object/goal and task/object/swap/lan.

6. Planned Experiments and Benchmark Metrics

Main comparisons. Use VLA-only execution, GPT review constrained to KEEP, unconditional GPT EEF correction, a heuristic gate with the same GPT corrector, and the full GPT gate–corrector–audit system. Add a GPT gate with a fixed geometric repair to isolate judgment from generation. An unrestricted GPT takeover is a separate capability/cost reference, not an equal-action-space baseline. Compare geometric edits, bounded programs, and prediction-assisted review under matched budgets.

Decision metrics. On paired continuations, report the rate of choosing CORRECT when $(Y_K, Y_C) = (0, 1)$ and KEEP when $(1, 0)$. Report the unresolved and equal-outcome fractions separately. In equal-success cases, unnecessary intervention incurs cost even when success is preserved. A paired candidate regret can be computed as $\max(Y_K, Y_C) - Y_b$, where Y_b is the selected branch outcome. This is a hindsight diagnostic, never an online selection signal or proof of optimality beyond those two candidates.

Correction metrics. Measure accepted program validity, actual EEF displacement and orientation change, correction duration, local recovery evidence, and final success change. Report harm on checkpoints where the original VLA succeeds. Rejected candidates, audit revisions, correction-limit saturation, and unexecuted proposals remain visible. Improvement from moving the EEF closer to a visually chosen point is not sufficient if it reduces final task success.

Experiment A: preserve versus rescue. On the natural panel, compare the hybrid’s paired success gain and intervention rate against the VLA. On the stress panel, separate local misalignment, grasp/placement mistakes, observation ambiguity, and failures beyond the correction envelope. A useful system should rescue some VLA failures while limiting newly introduced failures. Report both terms instead of only net improvement.

Experiment B: is audit useful? Compare no second-pass audit, same-model audit, and deterministic interface checks alone. Hold total model budgets fixed in one comparison and allow actual incurred costs in another. Evaluate whether the second pass catches bad edits or merely repeats the original rationale. GPT self-consistency is not independent verification; only executed outcomes establish intervention benefit.

Experiment C: correction scope and state freshness. Vary EEF edit magnitude, orientation freedom, repair duration, and revision count. Compare fresh re-observation and VLA re-query after edits with a labeled diagnostic that reuses the old proposal suffix. Separately inject review delay to quantify stale-action failure. This paper uses latency as a cost and failure factor; designing a new low-latency scheduler remains the second proposal’s objective.

Possible result	Defensible interpretation
More rescued failures than damaged successes, with bounded cost	Supports selective deployed correction over the frozen VLA.
Always-correct performs equally well	Binary review has not established an independent advantage.
Audit raises call cost without reducing harmful edits	The extra review pass is not justified under the measured budget.
Checkpoint gains disappear in closed-loop episodes	Intervention-induced state shift or stale handoffs remain unresolved.
Small geometric edits match generated programs	Simple correction may suffice; narrow the Code-as-Policy claim.

7. Statistical Protocol, Risks, and Expected Contribution

Primary evaluation. Preregister closed-loop task-success improvement at a fixed review/correction budget, with the harmful-intervention rate on baseline-successful checkpoints as a key secondary endpoint. Use paired seeds and intervals clustered by underlying task; repeated checkpoints from one rollout are not independent tasks. Report uncertainty for candidate-pair labels and candidate regret. Never return test-branch outcomes to the reviewer or use them to tune its gate. Benchmark construction and test outcome assignment remain evaluator-side operations.

Efficiency and reliability. Report prompt/completion/total tokens, VLM calls and unmetered calls, review latency, EEf correction time, recovery time, and total wall time per episode. Compare success–intervention–cost curves rather than assuming every extra review is valuable. Distinguish natural sampling from any balanced stress set. Count success only from the final simulator or independent harness. Provider, transport, image, resource, and interrupted records are excluded from valid-verdict task denominators and reported separately; implementation errors, rejection exhaustion, and missing coverage are also reported.

Principal failure modes. A reviewer can hallucinate a geometric error, confuse coordinate frames, or overrule a correct VLA. The same GPT used for generation and audit can reinforce its own mistake. A bounded correction can be locally plausible but incompatible with the next policy state. Sensing ambiguity can make reliable branch selection impossible. Each failure class needs retained images, visible rationale, action receipts, and final verdicts; hidden reasoning and simulator state are never exposed to the evaluated agent. Preserve original, corrected, failed, rejected, and diagnostic runs.

Why this is a benchmark contribution. Existing task success alone cannot reveal whether a reviewer knew when to intervene, whether its correction helped, or whether extra calls merely compensated for a poor base policy. Paired candidate continuations plus complete hybrid rollouts expose these distinctions. The benchmark should report candidate-dependent labels, natural prevalence, cross-VLA transfer, and latency conditions. It must not present privileged hindsight selection as a deployable competitor.

Optional learning extension. Correction traces may later support DAGger-style aggregation or VLA fine-tuning. If studied, evaluate the updated VLA *with GPT still active* under the same deployed architecture, and separate training gains from runtime correction gains. A GPT-free ablation can quantify component contribution, but removing GPT is neither the objective nor a requirement of this proposal. The main paper can be completed with frozen policy weights.

Expected paper claim. A persistent GPT reviewer can selectively retain VLA actions or improve them through audited EEf corrections, with measurable gains and identifiable limits. The method contribution is the binary review-and-correction interface; the experimental contribution is a benchmark that separates preserving correct behavior, repairing mistakes, and unnecessary or harmful intervention. This claim is independent of the first paper’s control representation and the second paper’s execution-timing mechanism.

References

- [1] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [2] π_0 : A Vision-Language-Action Flow Model for General Robot Control. 2024. <https://arxiv.org/abs/2410.24164>
- [3] Real-Time Execution of Action Chunking Flow Policies. 2025. <https://arxiv.org/abs/2506.07339>
- [4] Ross, Gordon, and Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. 2011. <https://arxiv.org/abs/1011.0686>
- [5] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>